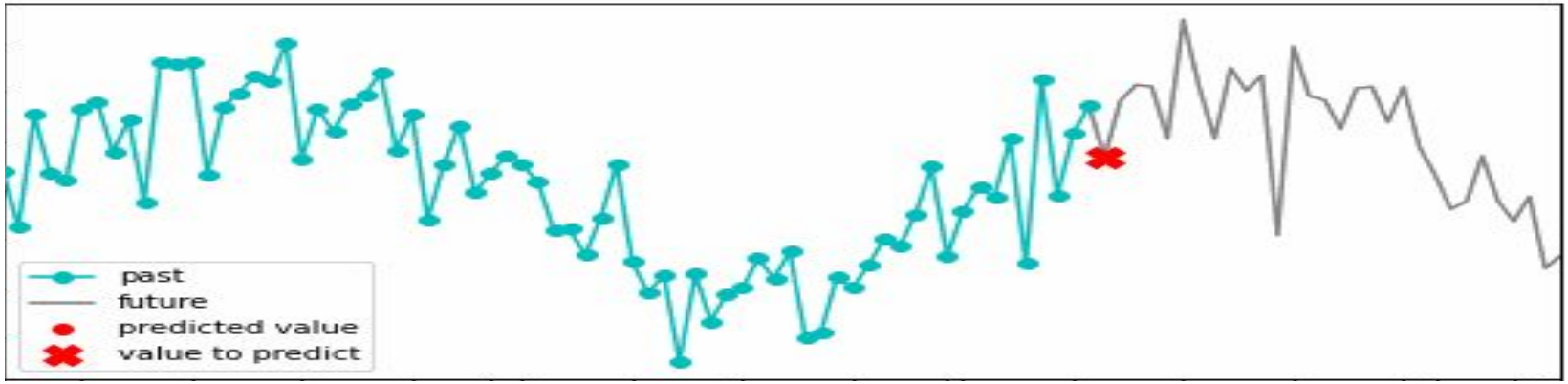
Universidad Autónoma de Cd. Juárez



Métodos para Pronóstico y Analítica Avanzada

TEMA 2: Modelos de Pronóstico Estadísticos

Dr. Rogelio Florencia Juárez

Maestría en Inteligencia Artificial y Analítica de Datos
MIAAD

DIEC
Departamento de Ingeniería Eléctrica y Computación UACJ

Contenido

- Métodos de suavizado
- Modelos de suavizamiento exponencial
- Modelos ARIMA (*Autoregressive Integrated Moving Average*)
- Métodos de descomposición de series temporales
- Modelos de tendencia y estacionalidad en series temporales

Métodos de suavizado

Métodos de suavizado

- Constituyen una de las familias fundamentales en el análisis y pronóstico de series de tiempo.

- No modelan explícitamente la estructura probabilística de los datos.
  - Reducen la variabilidad aleatoria (ruido) para revelar patrones subyacentes como el ***nivel*** (valor real que tiene la serie en un determinado momento, sin ruido), la ***tendencia*** o la ***estacionalidad***.

- Suponen que una serie puede interpretarse como una señal compuesta por ***una estructura sistemática*** más ***un componente aleatorio***
  - al suavizar la serie se atenúan las fluctuaciones erráticas sin destruir la información relevante.

Métodos de suavizado

Matemáticamente:

$$y_t = S_t + \varepsilon_t$$

donde:

y_t: valor observado de la serie en el tiempo t.

S_t: componente sistemático (señal). Representa el comportamiento estructurado y explicable de la serie.

ε_t: componente aleatorio (ruido). Representa:

- fluctuaciones impredecibles
- errores de medición
- shocks externos
- variabilidad no explicada por el modelo

Métodos de suavizado

Matemáticamente:

$$y_t = S_t + \varepsilon_t$$

donde:

y_t: valor observado de la serie en el tiempo t.

S_t: componente sistemático (señal). Representa el comportamiento estructurado y explicable de la serie.

ε_t: componente aleatorio (ruido). Representa:

- fluctuaciones impredecibles
- errores de medición
- shocks externos
- variabilidad no explicada por el modelo

Los métodos de suavizado buscan estimar:

$$\widehat{y_t} \approx S_t$$

ignorando deliberadamente el ruido ε_t.

- y_t: Valor real observado.
- $\widehat{y_t}$: Valor estimado (suavizado)
- S_t: Señal o nivel subyacente
- ε_t: Ruido aleatorio

Métodos de suavizado

Suavizar = filtrar el ruido para recuperar la señal.

Objetivo = entender el comportamiento de la serie y/o generar pronósticos de corto y mediano plazo.

Matemáticamente:

$$y_t = S_t +$$

donde:

y_t: valor observado de la serie en el tiempo t.

S_t: componente sistemático (señal). Representa el comportamiento estructurado y explicable de la serie.

ε_t: componente aleatorio (ruido). Representa:

- fluctuaciones impredecibles
- errores de medición
- shocks externos
- variabilidad no explicada por el modelo

Los métodos de suavizado buscan estimar:

$$\widehat{y_t} \approx S_t$$

ignorando deliberadamente el ruido ε_t.

- y_t: Valor real observado.
- $\widehat{y_t}$: Valor estimado (suavizado)
- S_t: Señal o nivel subyacente
- ε_t: Ruido aleatorio

Métodos de suavizado

- Media móvil simple (SMA, *Simple Moving Average*)
- Media móvil ponderada (WMA)
- Media móvil exponencial (EMA)

Métodos de suavizado (SMA)

- **Media móvil simple (SMA, *Simple Moving Average*)**
  - Suaviza la serie reemplazando cada valor por el promedio aritmético de los últimos k valores observados.
  - Reduce el ruido de corto plazo, pero ***todos los valores del pasado reciente pesan igual***, sin distinguir qué tan nuevos son.

$$\mathrm{SMA}_t = \frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$$

donde:

- y_{t-i}: observaciones pasadas.
- k: orden de la media móvil que determina el nivel de suavizado.

Métodos de suavizado (SMA)

Ejemplo:

$$\mathrm{SMA}_t = \frac{1}{k}\sum_{i=0}^{k-1} y_{t-i}$$

			SMA
1	10	-	-
2	12	-	-
3	13	(13 + 12 + 10) / 3	11.67
4	16	(16 + 13 + 12) / 3	13.67
5	15	(15 + 16 + 13) / 3	14.67
6	20	(20 + 15 + 16) / 3	17
7	22	(22 + 20 + 15) / 3	19
8	21	(21 + 22 + 20) / 3	21
9	23	(23 + 21 + 22) / 3	22
10	25	(25 + 23 + 21) / 3	23
11	26	(26 + 25 + 23) / 3	24.67
12	28	(28 + 26 + 25) / 3	26.33
13	30	(30 + 28 + 26) / 3	28
14	29	(29 + 30 + 28) / 3	29
15	31	(31 + 29 + 30) / 3	30
16	33	(33 + 31 + 29) / 3	31
17	35	(35 + 33 + 31) / 3	33
18	36	(36 + 35 + 33) / 3	34.67
19	38	(38 + 36 + 35) / 3	36.33
20	42	(42 + 38 + 36) / 3	38.67

Métodos de suavizado (WMA)

- **Media móvil ponderada (WMA, *Weighted Moving Average*)**
  - Extensión de la media móvil simple en la que ***no todos los datos tienen la misma importancia***.
  - En lugar de promediar de forma uniforme, se asignan ***pesos diferentes***, normalmente mayores a los valores más recientes (datos recientes contienen más información sobre el futuro).

$$\mathrm{WMA}_t = \sum_{i=0}^{k-1} w_i \; y_{t-i} \quad \text{con} \quad \sum w_i = 1$$

donde:

- y_{t-i}: observaciones pasadas.
- w_i: pesos.
- $w_0 > w_1 > \cdots > w_{\text{k-1}}$

Métodos de suavizado (WMA)

Ejemplo:

$$\mathrm{WMA}_t = \sum_{i=0}^{k-1} w_i \; y_{t-i}$$

$w_0 = 0.5$
$w_1 = 0.3$
$w_2 = 0.2$

$$\sum w_i = 1$$

			WMA
1	10	-	-
2	12	-	-
3	13	0.5*13 + 0.3*12 + 0.2*10	12.1
4	16	0.5*16 + 0.3*13 + 0.2*12	14.3
5	15	0.5*15 + 0.3*16 + 0.2*13	14.9
6	20	0.5*20 + 0.3*15 + 0.2*16	17.7
7	22	0.5*22 + 0.3*20 + 0.2*15	20
8	21	0.5*21 + 0.3*22 + 0.2*20	21.1
9	23	0.5*23 + 0.3*21 + 0.2*22	22.2
10	25	0.5*25 + 0.3*23 + 0.2*21	23.6
11	26	0.5*26 + 0.3*25 + 0.2*23	25.1
12	28	0.5*28 + 0.3*26 + 0.2*25	26.8
13	30	0.5*30 + 0.3*28 + 0.2*26	28.6
14	29	0.5*29 + 0.3*30 + 0.2*28	29.1
15	31	0.5*31 + 0.3*29 + 0.2*30	30.2
16	33	0.5*33 + 0.3*31 + 0.2*29	31.6
17	35	0.5*35 + 0.3*33 + 0.2*31	33.6
18	36	0.5*36 + 0.3*35 + 0.2*33	35.1
19	38	0.5*38 + 0.3*36 + 0.2*35	36.8
20	42	0.5*42 + 0.3*38 + 0.2*36	39.6

Métodos de suavizado (EMA)

- **Media móvil exponencial (EMA, *Exponential Moving Average*)**
  - Asigna pesos que decrecen exponencialmente conforme los datos son más antiguos.
  - Es la base del suavizamiento exponencial simple (SES). Todos los datos influyen, pero los más recientes pesan mucho más.

$$S_t = \alpha y_t + (1 - \alpha) S_{t-1}$$

donde:

- $0 < \alpha < 1$: observaciones pasadas.
- S_t: valor suavizado.
- α : parámetro de suavizamiento.

Métodos de suavizado (EMA)

- **Media móvil exponencial (EMA, *Exponential M***
  - Asigna pesos que decrecen exponencial
    antiguos.
  - Es la base del suavizamiento exponencial
    pero los más recientes pesan mucho más.

$$S_t = \alpha y_t + (1 - \alpha$$

donde:

- $0 < \alpha < 1$: observaciones pasadas.
- S_t: valor suavizado.
- α : parámetro de suavizamiento.

α cercano a 0 (ej. 0.1)

- el pasado pesa mucho
- el modelo es muy suave
- cambia lentamente
- reacciona tarde a cambios

Útil cuando:

- la serie es estable
- hay poco ruido
- no hay cambios bruscos

Métodos de suavizado (EMA)

- **Media móvil exponencial (EMA, *Exponential M***
  - Asigna pesos que decrecen exponencial antiguos.
  - Es la base del suavizamiento exponencial pero los más recientes pesan mucho más.

$$S_t = \alpha y_t + (1 - \alpha$$

donde:

- $0 < \alpha < 1$: observaciones pasadas.
- S_t: valor suavizado.
- α : parámetro de suavizamiento.

α cercano a 0 (ej. 0.1)

- el pasado pesa mucho
- el modelo es muy suave
- cambia lentamente
- reacciona tarde a cambios

Útil cuando:

- la serie es estable
- hay poco ruido
- no hay cambios bruscos

α cercano a 1 (ej. 0.8)

- el dato actual pesa mucho
- reacciona rápido
- sigue cambios abruptos
- es más ruidoso

Útil cuando:

- la serie cambia rápido
- hay choques o rupturas
- se requiere reacción inmediata

Métodos de suavizado (EMA)

Ejemplo:

$$S_t = \alpha y_t + (1-\alpha)S_{t-1}$$

Definición **recursiva**

El valor suavizado actual (S_t) es una combinación del dato actual (y_t) y del valor suavizado anterior (S_{t-1}).

			EMA
1	10	Inicial	10
2	12	0.3*12 + 0.7*10	10.6
3	13	0.3*13 + 0.7*10.6	11.32
4	16	0.3*16 + 0.7*11.32	12.72
5	15	0.3*15 + 0.7*12.72	13.4
6	20	0.3*20 + 0.7*13.41	15.39
7	22	0.3*22 + 0.7*15.38	17.37
8	21	0.3*21 + 0.7*17.37	18.46
9	23	0.3*23 + 0.7*18.46	19.82
10	25	0.3*25 + 0.7*19.82	21.37
11	26	0.3*26 + 0.7*21.37	22.76
12	28	0.3*28 + 0.7*22.76	24.33
13	30	0.3*30 + 0.7*24.33	26.03
14	29	0.3*29 + 0.7*26.03	26.92
15	31	0.3*31 + 0.7*26.92	28.14
16	33	0.3*33 + 0.7*28.15	29.61
17	35	0.3*35 + 0.7*29.6	31.22
18	36	0.3*36 + 0.7*31.22	32.65
19	38	0.3*38 + 0.7*32.66	34.26
20	42	0.3*42 + 0.7*34.26	36.58

Métodos de suavizado (EMA)

Ejemplo:

$$S_t = \alpha y_t + (1-\alpha)\boxed{S_{t-1}}$$

$$S_{t-1} = \alpha y_{t-1} + (1-\alpha)S_{t-2}$$

			EMA
1	10	Inicial	10
2	12	0.3*12 + 0.7*10	10.6
3	13	0.3*13 + 0.7*10.6	11.32
4	16	0.3*16 + 0.7*11.32	12.72
5	15	0.3*15 + 0.7*12.72	13.4
6	20	0.3*20 + 0.7*13.41	15.39
7	22	0.3*22 + 0.7*15.38	17.37
8	21	0.3*21 + 0.7*17.37	18.46
9	23	0.3*23 + 0.7*18.46	19.82
10	25	0.3*25 + 0.7*19.82	21.37
11	26	0.3*26 + 0.7*21.37	22.76
12	28	0.3*28 + 0.7*22.76	24.33
13	30	0.3*30 + 0.7*24.33	26.03
14	29	0.3*29 + 0.7*26.03	26.92
15	31	0.3*31 + 0.7*26.92	28.14
16	33	0.3*33 + 0.7*28.15	29.61
17	35	0.3*35 + 0.7*29.6	31.22
18	36	0.3*36 + 0.7*31.22	32.65
19	38	0.3*38 + 0.7*32.66	34.26
20	42	0.3*42 + 0.7*34.26	36.58

Métodos de suavizado (EMA)

Ejemplo:

$$S_t = \alpha y_t + (1-\alpha)\boxed{S_{t-1}}$$

$$S_{t-1} = \overbrace{\alpha y_{t-1} + (1-\alpha)S_{t-2}}$$

Sustituyendo:

$$S_t = \alpha y_t + (1-\alpha)[\alpha y_{t-1} + (1-\alpha)S_{t-2}]$$

			EMA
1	10	Inicial	10
2	12	0.3*12 + 0.7*10	10.6
3	13	0.3*13 + 0.7*10.6	11.32
4	16	0.3*16 + 0.7*11.32	12.72
5	15	0.3*15 + 0.7*12.72	13.4
6	20	0.3*20 + 0.7*13.41	15.39
7	22	0.3*22 + 0.7*15.38	17.37
8	21	0.3*21 + 0.7*17.37	18.46
	23	0.3*23 + 0.7*18.46	19.82
	25	0.3*25 + 0.7*19.82	21.37
11	26	0.3*26 + 0.7*21.37	22.76
12	28	0.3*28 + 0.7*22.76	24.33
13	30	0.3*30 + 0.7*24.33	26.03
14	29	0.3*29 + 0.7*26.03	26.92
15	31	0.3*31 + 0.7*26.92	28.14
16	33	0.3*33 + 0.7*28.15	29.61
17	35	0.3*35 + 0.7*29.6	31.22
18	36	0.3*36 + 0.7*31.22	32.65
19	38	0.3*38 + 0.7*32.66	34.26
20	42	0.3*42 + 0.7*34.26	36.58

Métodos de suavizado (EMA)

Ejemplo:

$$S_t = \alpha y_t + (1-\alpha)\boxed{S_{t-1}}$$

$$S_{t-1} = \alpha y_{t-1} + (1-\alpha)S_{t-2}$$

Sustituyendo:

$$S_t = \alpha y_t + (1-\alpha)[\alpha y_{t-1} + (1-\alpha)S_{t-2}]$$

Distribuyendo:

$$S_t = \alpha y_t + \alpha(1-\alpha)y_{t-1} + (1-\alpha)(1-\alpha)S_{t-2}$$

$$S_t = \alpha y_t + \alpha(1-\alpha)y_{t-1} + (1-\alpha)^2 S_{t-2}$$

			EMA
1	10	Inicial	10
2	12	0.3*12 + 0.7*10	10.6
3	13	0.3*13 + 0.7*10.6	11.32
4	16	0.3*16 + 0.7*11.32	12.72
5	15	0.3*15 + 0.7*12.72	13.4
6	20	0.3*20 + 0.7*13.41	15.39
7	22	0.3*22 + 0.7*15.38	17.37
8	21	0.3*21 + 0.7*17.37	18.46
	23	0.3*23 + 0.7*18.46	19.82
	25	0.3*25 + 0.7*19.82	21.37
11	26	0.3*26 + 0.7*21.37	22.76
12	28	0.3*28 + 0.7*22.76	24.33
13	30	0.3*30 + 0.7*24.33	26.03
		0.3*29 + 0.7*26.03	26.92
		0.3*31 + 0.7*26.92	28.14
		0.3*33 + 0.7*28.15	29.61
17	35	0.3*35 + 0.7*29.6	31.22
18	36	0.3*36 + 0.7*31.22	32.65
19	38	0.3*38 + 0.7*32.66	34.26
20	42	0.3*42 + 0.7*34.26	36.58

Métodos de suavizado (EMA)

Ejemplo:

$$S_t = \alpha y_t + (1-\alpha)\boxed{S_{t-1}}$$

$$S_{t-1} = \overbrace{\alpha y_{t-1} + (1-\alpha)S_{t-2}}$$

Sustituyendo:

$$S_t = \alpha y_t + (1-\alpha)[\alpha y_{t-1} + (1-\alpha)S_{t-2}]$$

Distribuyendo:

$$S_t = \alpha y_t + \alpha(1-\alpha)y_{t-1} + (1-\alpha)(1-\alpha)S_{t-2}$$

$$S_t = \alpha y_t + \alpha(1-\alpha)y_{t-1} + (1-\alpha)^2 S_{t-2}$$

Etc…

$$S_t = \alpha y_t + \alpha(1-\alpha)y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \alpha(1-\alpha)^3 y_{t-3} + \cdots$$

			EMA
1	10	Inicial	10
2	12	0.3*12 + 0.7*10	10.6
3	13	0.3*13 + 0.7*10.6	11.32
4	16	0.3*16 + 0.7*11.32	12.72
5	15	0.3*15 + 0.7*12.72	13.4
6	20	0.3*20 + 0.7*13.41	15.39
7	22	0.3*22 + 0.7*15.38	17.37
8	21	0.3*21 + 0.7*17.37	18.46
	23	0.3*23 + 0.7*18.46	19.82
	25	0.3*25 + 0.7*19.82	21.37
11	26	0.3*26 + 0.7*21.37	22.76
12	28	0.3*28 + 0.7*22.76	24.33
13	30	0.3*30 + 0.7*24.33	26.03
		0.3*29 + 0.7*26.03	26.92
		0.3*31 + 0.7*26.92	28.14
		0.3*33 + 0.7*28.15	29.61
17	35	0.3*35 + 0.7*29.6	31.22
18	36	0.3*36 + 0.7*31.22	32.65
19	38	0.3*38 + 0.7*32.66	34.26
		34.26	36.58

Métodos de suavizado (EMA)

Ejemplo:

1	10	0.000342	0.003419669
2	12	0.000489	0.005862289
3	13	0.000698	0.00907259
4	16	0.000997	0.015951807
5	15	0.001424	0.021364027
6	20	0.002035	0.040693384
7	22	0.002907	0.063946747
8	21	0.004152	0.087200109
9	23	0.005932	0.136435545
10	25	0.008474	0.211856437
11	26	0.012106	0.314758135
12	28	0.017294	0.484243284
13	30	0.024706	0.7411887
14	29	0.035295	1.0235463
15	31	0.050421	1.563051
16	33	0.07203	2.37699
17	35	0.1029	3.6015
18	36	0.147	5.292
19	38	0.21	7.98
20	42	0.3	12.6

36.57308002

Métodos de suavizado (SMA, WMA, EMA)

Método	Pesos	Interpretación
SMA	Iguales	Todos los datos valen lo mismo
WMA	Definidos por el usuario	Algunos datos pesan más
EMA	Automáticos	Peso decrece exponencialmente

Implementación

índice

```
import numpy as np
import pandas as pd


# Serie de tiempo
data = [10, 12, 13, 16, 15, 20, 22, 21, 23, 25,
        26, 28, 30, 29, 31, 33, 35, 36, 38, 42]

serie = pd.Series(data)
```

Convierte la lista en una serie temporal

0	10
1	12
2	13
3	16
4	15
5	20
6	22
7	21
8	23
9	25
10	26
11	28
12	30
13	29
14	31
15	33
16	35
17	36
18	38
19	42

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()


#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)


#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

- Crea una ventana deslizante de tamaño k
- Para cada punto toma los últimos k valores

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()
```

Calcula el promedio de la ventana de datos

```
#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)


#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()


#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)


#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

Define los pesos

Dato	Peso
y_{-2}	0.2
y_{-1}	0.3
y	0.5

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()
```

```
#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)
```

Función **wma**:

- values: ventana de datos.
- producto punto:

$$0.2 \cdot 13 + 0.3 \cdot 16 + 0.5 \cdot 15$$

```
#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()


#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)


#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

Aplica la función **wma** a la ventana de datos.

Implementación

```
#1. Media Móvil Simple (SMA)
k = 3
sma = serie.rolling(window=k).mean()


#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(values):
    return np.dot(values, weights)

wma_series = serie.rolling(window=3).apply(wma, raw=True)


#3. Media Móvil Exponencial (EMA)
alpha = 0.3
ema = serie.ewm(alpha=alpha, adjust=False).mean()
```

Calcula de forma recursiva:

$$S_t = \alpha y_t + (1-\alpha)S_{t-1}$$

Implementación

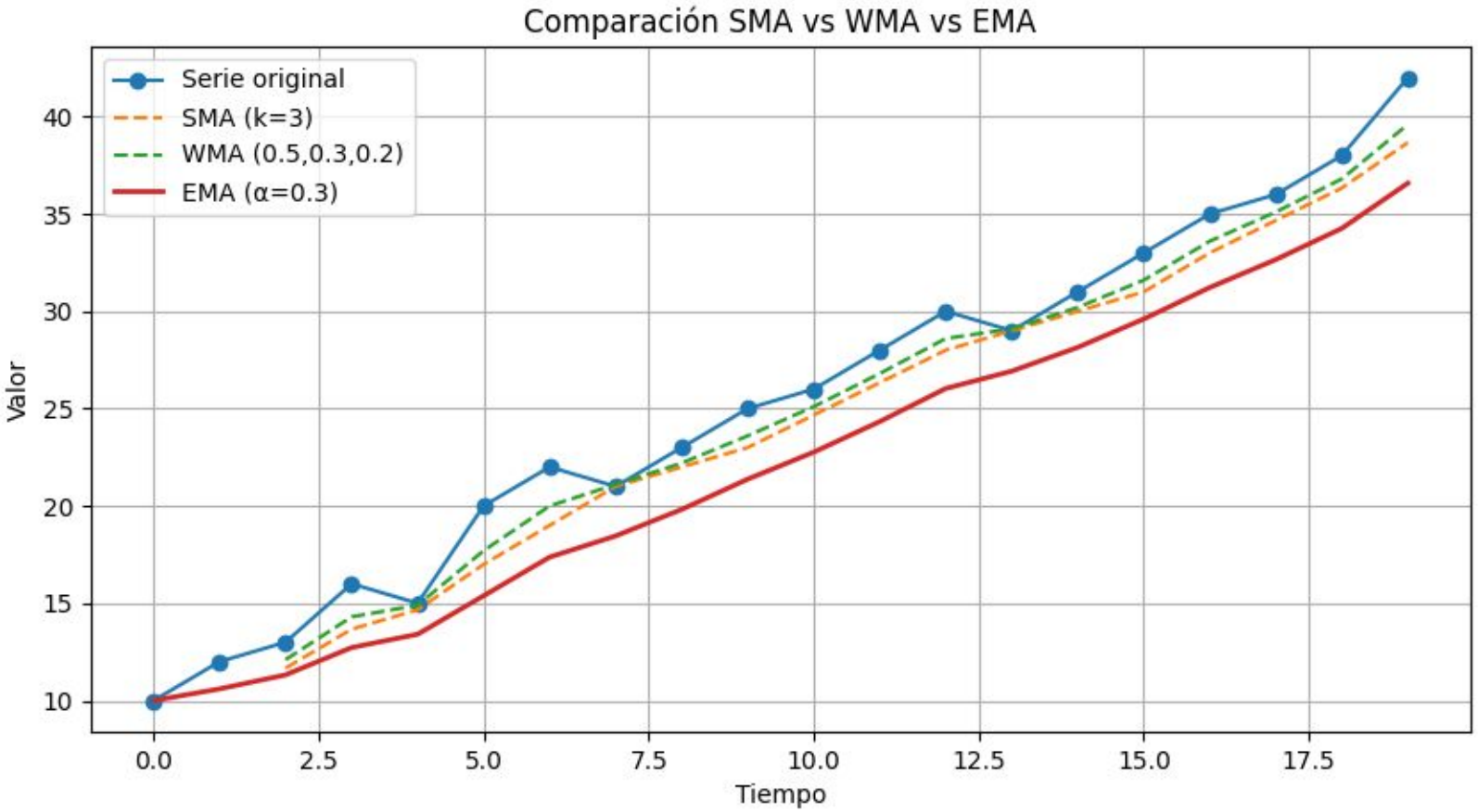
```
#Resultados
df = pd.DataFrame({
    "y": serie,
    "SMA": sma,
    "WMA": wma_series,
    "EMA": ema
})

print(df)
```

	y	SMA	WMA	EMA
0	10	NaN	NaN	10.000000
1	12	NaN	NaN	10.600000
2	13	11.666667	12.1	11.320000
3	16	13.666667	14.3	12.724000
4	15	14.666667	14.9	13.406800
5	20	17.000000	17.7	15.384760
6	22	19.000000	20.0	17.369332
7	21	21.000000	21.1	18.458532
8	23	22.000000	22.2	19.820973
9	25	23.000000	23.6	21.374681
10	26	24.666667	25.1	22.762277
11	28	26.333333	26.8	24.333594
12	30	28.000000	28.6	26.033516
13	29	29.000000	29.1	26.923461
14	31	30.000000	30.2	28.146423
15	33	31.000000	31.6	29.602496
16	35	33.000000	33.6	31.221747
17	36	34.666667	35.1	32.655223
18	38	36.333333	36.8	34.258656
19	42	38.666667	39.6	36.581059

Implementación

```
#Gráfica
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(serie, label="Serie original", marker="o")
plt.plot(sma, label="SMA (k=3)", linestyle="--")
plt.plot(wma_series, label="WMA (0.5,0.3,0.2)", linestyle="--")
plt.plot(ema, label="EMA (α=0.3)", linewidth=2)

plt.legend()
plt.title("Comparación SMA vs WMA vs EMA")
plt.xlabel("Tiempo")
plt.ylabel("Valor")
plt.grid(True)
plt.show()
```

Implementación



Implementación - Pronósticos

```
import numpy as np
import pandas as pd


def metrics(y_true, y_pred):
    mask = ~np.isnan(y_pred)
    y_true = y_true[mask]
    y_pred = y_pred[mask]

    mae = np.mean(np.abs(y_true - y_pred))
    mse = np.mean((y_true - y_pred) ** 2)
    rmse = np.sqrt(mse)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    smape = np.mean(
        2 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.abs(y_pred))
    ) * 100

    return mae, mse, rmse, mape, smape
```

Implementación - Pronósticos

```
# Serie de tiempo
data = [10, 12, 13, 16, 15, 20, 22, 21, 23, 25,
        26, 28, 30, 29, 31, 33, 35, 36, 38, 42]
serie = pd.Series(data)


#1. Media Móvil Simple (SMA)
sma = serie.rolling(window=3).mean()
sma_forecast = sma.shift(1)


#2. Media Móvil Ponderada (WMA)
weights = np.array([0.2, 0.3, 0.5])

def wma(x):
    return np.dot(x, weights)

wma_series = serie.rolling(3).apply(wma, raw=True)
wma_forecast = wma_series.shift(1)

#3. Media Móvil Exponencial (EMA)
alpha = 0.99
ema = serie.ewm(alpha=alpha, adjust=False).mean()
ema_forecast = ema.shift(1)
```

El valor suavizado en $t-1$ se usa para pronosticar t.

Implementación - Pronósticos

```
#Resultados
df = pd.DataFrame({
    "y": serie,
    "SMA": sma_forecast,
    "WMA": wma_forecast,
    "EMA": ema_forecast
})

print(df)


#Gráfica
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(serie, label="Serie original", marker="o")
plt.plot(sma_forecast, label="SMA (k=3)", linestyle="--")
plt.plot(wma_forecast, label="WMA (0.5,0.3,0.2)", linestyle="--")
plt.plot(ema_forecast, label="EMA (α=0.3)", linewidth=2)

plt.legend()
plt.title("Comparación pronósticos SMA vs WMA vs EMA")
plt.xlabel("Tiempo")
plt.ylabel("Valor")
plt.grid(True)
plt.show()
```

Implementación - Pronósticos

```
results = pd.DataFrame({
    "SMA": metrics(serie, sma_forecast),
    "WMA": metrics(serie, wma_forecast),
    "EMA": metrics(serie, ema_forecast)
}, index=["MAE", "MSE", "RMSE", "MAPE", "sMAPE"])

print(results)
```

Implementación - Pronósticos

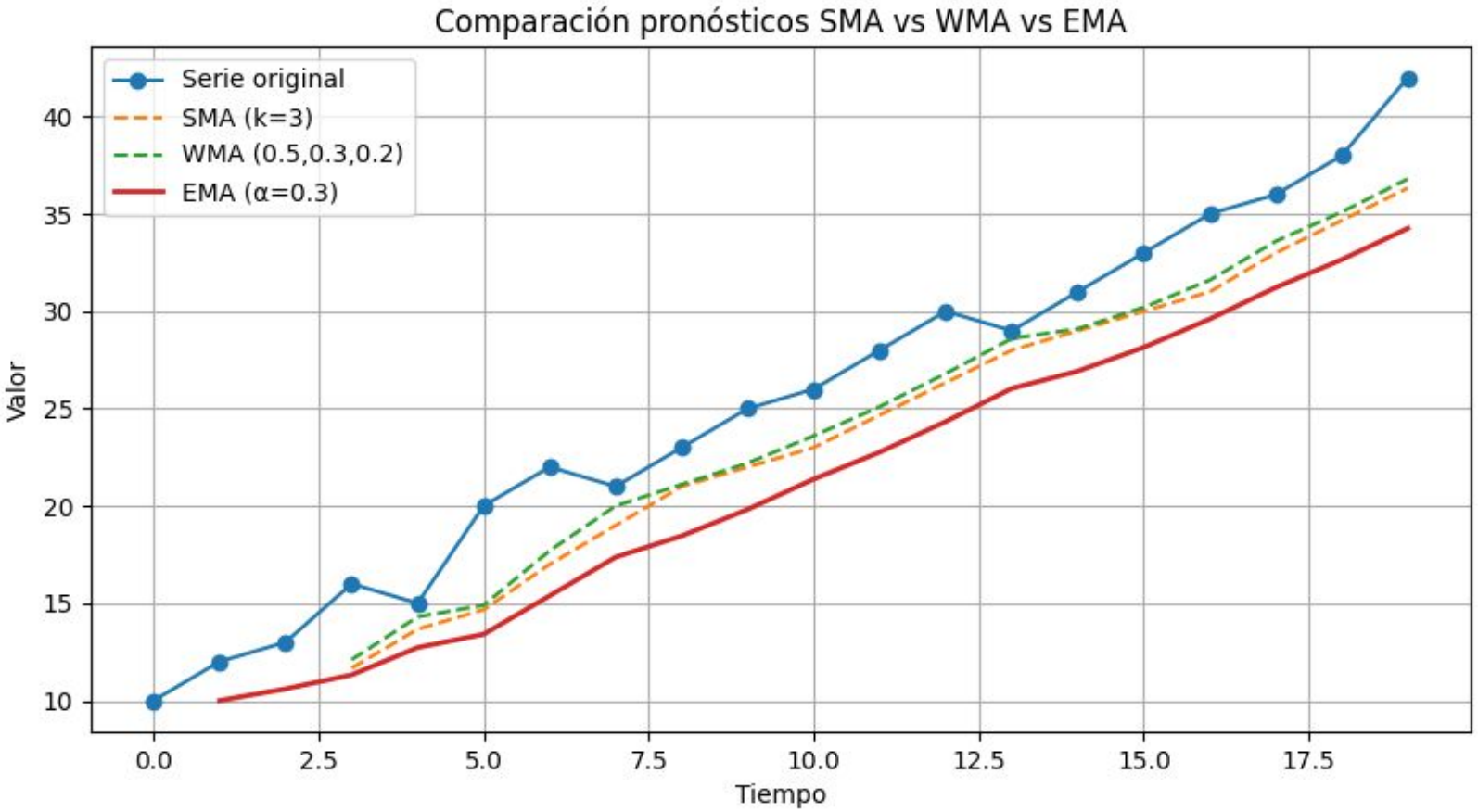
	y	SMA	WMA	EMA
0	10	NaN	NaN	NaN
1	12	NaN	NaN	10.000000
2	13	NaN	NaN	10.600000
3	16	11.666667	12.1	11.320000
4	15	13.666667	14.3	12.724000
5	20	14.666667	14.9	13.406800
6	22	17.000000	17.7	15.384760
7	21	19.000000	20.0	17.369332
8	23	21.000000	21.1	18.458532
9	25	22.000000	22.2	19.820973
10	26	23.000000	23.6	21.374681
11	28	24.666667	25.1	22.762277
12	30	26.333333	26.8	24.333594
13	29	28.000000	28.6	26.033516
14	31	29.000000	29.1	26.923461
15	33	30.000000	30.2	28.146423
16	35	31.000000	31.6	29.602496
17	36	33.000000	33.6	31.221747
18	38	34.666667	35.1	32.655223
19	42	36.333333	36.8	34.258656

Implementación

	y	SMA	WMA	EMA
0	10	NaN	NaN	10.000000
1	12	NaN	NaN	10.600000
2	13	11.666667	12.1	11.320000
3	16	13.666667	14.3	12.724000
4	15	14.666667	14.9	13.406800
5	20	17.000000	17.7	15.384760
6	22	19.000000	20.0	17.369332
7	21	21.000000	21.1	18.458532
8	23	22.000000	22.2	19.820973
9	25	23.000000	23.6	21.374681
10	26	24.666667	25.1	22.762277
11	28	26.333333	26.8	24.333594
12	30	28.000000	28.6	26.033516
13	29	29.000000	29.1	26.923461
14	31	30.000000	30.2	28.146423
15	33	31.000000	31.6	29.602496
16	35	33.000000	33.6	31.221747
17	36	34.666667	35.1	32.655223
18	38	36.333333	36.8	34.258656
19	42	38.666667	39.6	36.581059

	y	SMA	WMA	EMA
0	10	NaN	NaN	NaN
1	12	NaN	NaN	10.000000
2	13	NaN	NaN	10.600000
3	16	11.666667	12.1	11.320000
4	15	13.666667	14.3	12.724000
5	20	14.666667	14.9	13.406800
6	22	17.000000	17.7	15.384760
7	21	19.000000	20.0	17.369332
8	23	21.000000	21.1	18.458532
9	25	22.000000	22.2	19.820973
10	26	23.000000	23.6	21.374681
11	28	24.666667	25.1	22.762277
12	30	26.333333	26.8	24.333594
13	29	28.000000	28.6	26.033516
14	31	29.000000	29.1	26.923461
15	33	30.000000	30.2	28.146423
16	35	31.000000	31.6	29.602496
17	36	33.000000	33.6	31.221747
18	38	34.666667	35.1	32.655223
19	42	36.333333	36.8	34.258656

Implementación



Implementación

	SMA	WMA	EMA
MAE	3.235294	2.776471	4.663344
MSE	12.163399	9.496471	23.951261
RMSE	3.487606	3.081634	4.894003
MAPE	12.486339	10.644185	18.684196
sMAPE	13.586714	11.497856	20.848227

El mejor es WMA

Implementación - Ajuste alfa en EMA

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt



def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred)**2))


data = [10, 12, 13, 16, 15, 20, 22, 21, 23, 25,
        26, 28, 30, 29, 31, 33, 35, 36, 38, 42]

serie = pd.Series(data)
```

Implementación - Ajuste alfa en EMA

```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

```
array([0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1 , 0.11,
       0.12, 0.13, 0.14, 0.15, 0.16, 0.17, 0.18, 0.19, 0.2 , 0.21, 0.22,
       0.23, 0.24, 0.25, 0.26, 0.27, 0.28, 0.29, 0.3 , 0.31, 0.32, 0.33,
       0.34, 0.35, 0.36, 0.37, 0.38, 0.39, 0.4 , 0.41, 0.42, 0.43, 0.44,
       0.45, 0.46, 0.47, 0.48, 0.49, 0.5 , 0.51, 0.52, 0.53, 0.54, 0.55,
       0.56, 0.57, 0.58, 0.59, 0.6 , 0.61, 0.62, 0.63, 0.64, 0.65, 0.66,
       0.67, 0.68, 0.69, 0.7 , 0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77,
       0.78, 0.79, 0.8 , 0.81, 0.82, 0.83, 0.84, 0.85, 0.86, 0.87, 0.88,
       0.89, 0.9 , 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.98, 0.99])
```

Implementación - Ajuste alfa en EMA

```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

Empieza en 1 porque el primer valor no tiene pronóstico

Implementación - Ajuste alfa en EMA

```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

Guarda el error correspondiente a ese α.

Implementación - Ajuste alfa en EMA

```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

Guarda el error correspondiente a ese α.

```
[16.942681776750934,
 15.855936663898229,
 14.869374589505144,
 13.97260227171929,
 13.156348627780256,
 12.412344951876843,
 11.733217207220598,
 11.112389305492538,
 10.54399634909952,
 10.02280690214476,
```

Implementación - Ajuste alfa en EMA

```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

Índice del menor error para obtener de ***alphas*** el valor de α correspondiente.

Implementación - Ajuste alfa en EMA

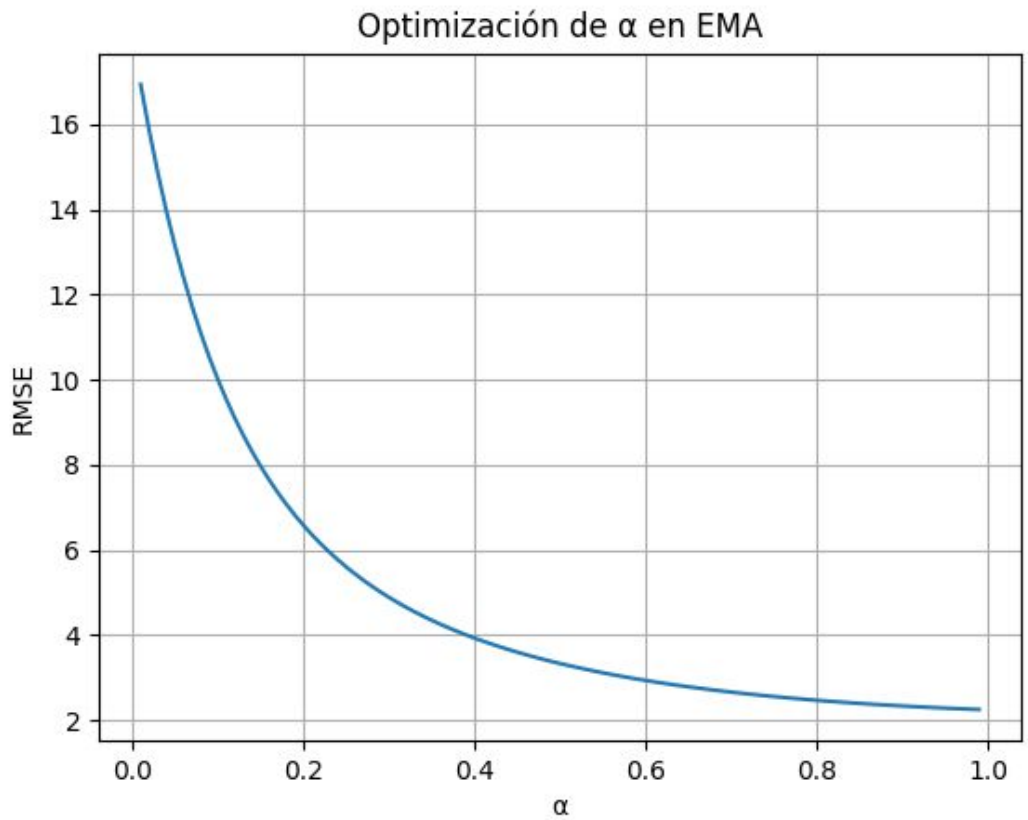
```
alphas = np.arange(0.01, 1.0, 0.01)
errors = []

for a in alphas:
    ema = serie.ewm(alpha=a, adjust=False).mean()
    forecast = ema.shift(1)

    error = rmse(serie[1:], forecast[1:])
    errors.append(error)

best_alpha = alphas[np.argmin(errors)]
best_error = min(errors)

print("Mejor alfa: " + str(best_alpha))
print("Mejor error: " + str(best_error))
```

Valor más pequeño de los errores.

```
Mejor alfa: 0.99
Mejor error: 2.2552342419926057
```

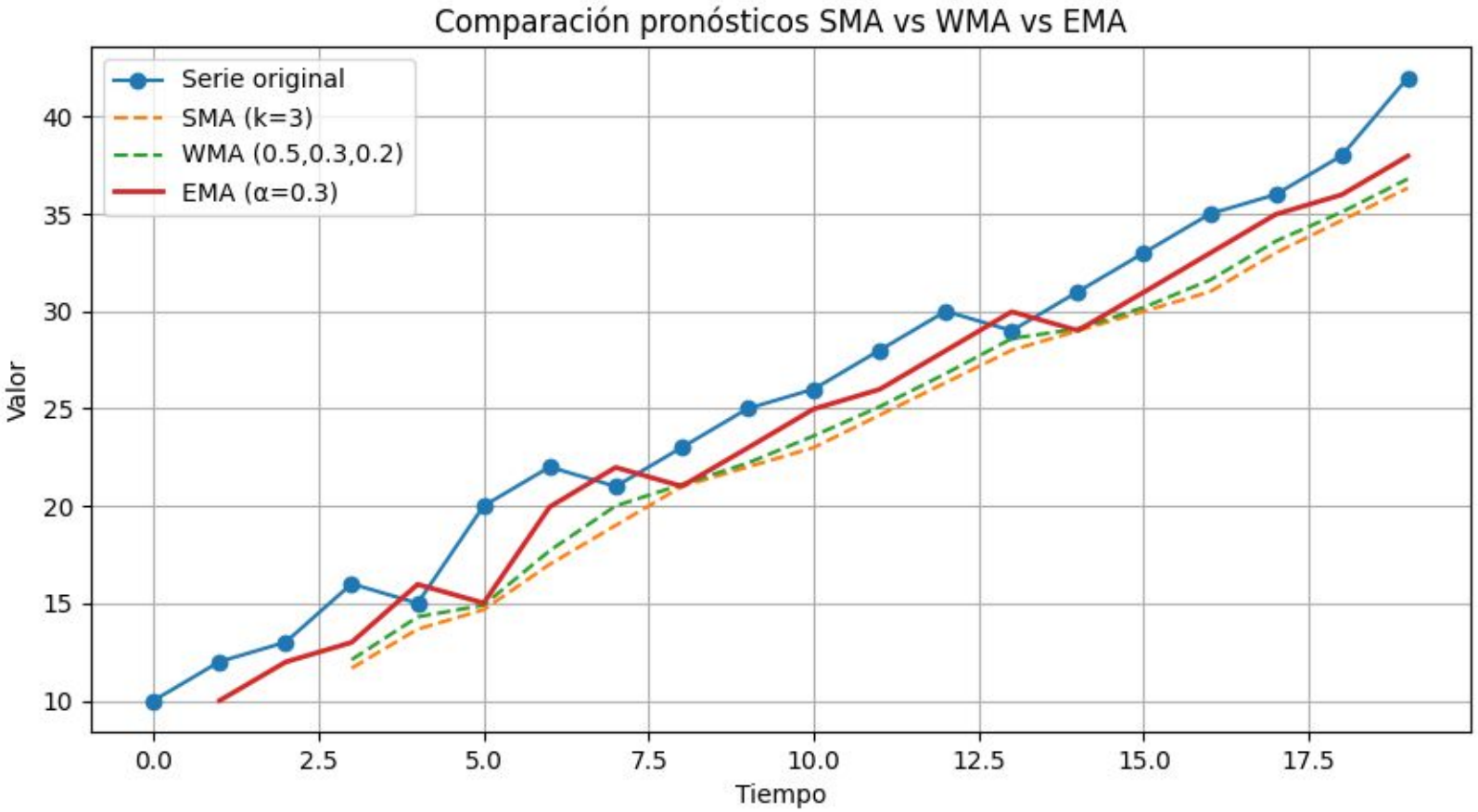
Implementación - Ajuste alfa en EMA

```
plt.plot(alphas, errors)
plt.xlabel("α")
plt.ylabel("RMSE")
plt.title("Optimización de α en EMA")
plt.grid(True)
plt.show()
```



Implementación - Ajuste alfa en EMA

EMA con α = 0.99



Implementación - Ajuste alfa en EMA

	SMA	WMA	EMA
MAE	3.235294	2.776471	4.663344
MSE	12.163399	9.496471	23.951261
RMSE	3.487606	3.081634	4.894003
MAPE	12.486339	10.644185	18.684196
sMAPE	13.586714	11.497856	20.848227

El mejor es WMA

$\alpha = 0.3$

	SMA	WMA	EMA
MAE	3.235294	2.776471	2.007422
MSE	12.163399	9.496471	5.086081
RMSE	3.487606	3.081634	2.255234
MAPE	12.486339	10.644185	8.562246
sMAPE	13.586714	11.497856	9.088100

El mejor es WMA

$\alpha = 0.99$

Ejercicio:

Considere los siguientes cuatro conjuntos de datos:

Conjunto 1:
[10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]

Conjunto 2:
[20, 21, 19, 20, 22, 21, 20, 19, 21, 20, 22, 21, 20, 19, 21, 20, 22, 21, 20, 19]

Conjunto 3:
[10, 13, 12, 15, 14, 17, 16, 19, 18, 21, 20, 23, 22, 25, 24, 27, 26, 29, 28, 31]

Conjunto 4:
[20, 25, 20, 25, 20, 25, 20, 25, 20, 25, 20, 25, 20, 25, 20, 25, 20, 25, 20, 25]

Ejercicio:

Actividad 1:

1. Aplica los métodos SMA, WMA y EMA a las cuatro series temporales para suavizar la serie.
2. Grafica los datos reales y los datos suavizados de los métodos.
3. Describa el comportamiento de cada método en cada una de las series.

Actividad 2:

1. Utilice los mismos métodos pero ahora como modelos ingenuos de pronóstico a un paso.
2. Calcule MAE, RMSE, MAPE y sMAPE.
3. Grafica los datos reales y los datos pronosticados por los métodos.
4. Compare los resultados y responde lo siguiente por cada una de las serie:
   - ✔ ¿Qué método responde mejor?

Redacta tus conclusiones generales sobre los métodos con respecto a las dos actividades.

Ejercicio:

Entregables:

- Subir documento de Word con los análisis de cada actividad y las conclusiones generales.

- Subir códigos de programación.

Modelos de suavizamiento exponencial

Modelos ARIMA (Autoregressive Integrated Moving Average)

Métodos de descomposición de series temporales

Modelos de tendencia y estacionalidad en series temporales

Preguntas